

# Introduction to neural networks

## Simple neural network

**Juan Irving Vásquez**  
jivg.org

Centro de Innovación y Desarrollo Tecnológico en Cómputo  
Instituto Politécnico Nacional

February 26, 2026©



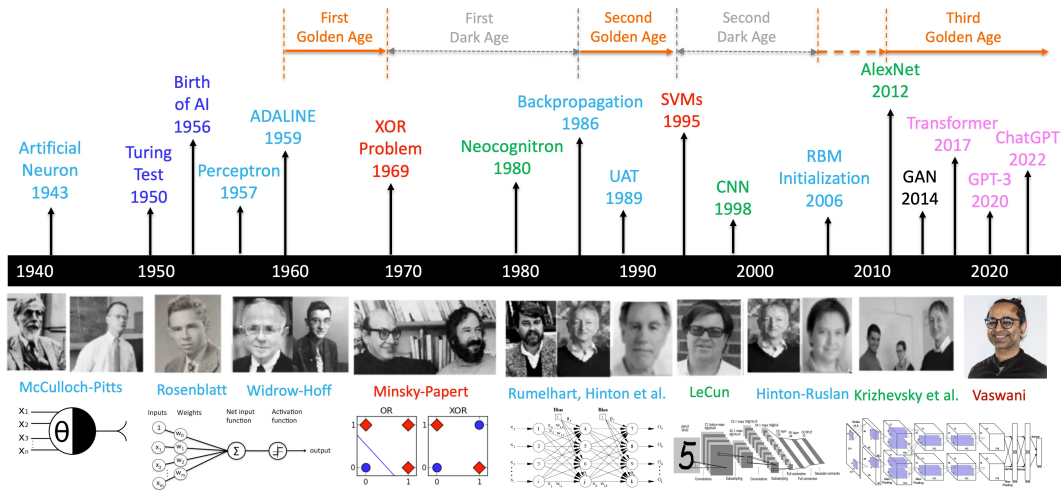
# Contents

- 1 Modern Artificial Neuron
- 2 Weight learning
- 3 Simple Network with multiple outputs
- 4 Practical aspects

# Roadmap

- 1 Modern Artificial Neuron
- 2 Weight learning
- 3 Simple Network with multiple outputs
- 4 Practical aspects

# The big picture <sup>1</sup>



<sup>1</sup><https://medium.com/@lmpo/a-brief-history-of-ai-with-deep-learning-26f7948bc87b>

# Evolution: Rosenblatt Perceptron vs. Modern Neuron

## The Rosenblatt Perceptron

- **Heaviside Step Function:** Output is strictly binary  $y \in \{0, 1\}$ .
- **Learning Rule:** Based on a simple heuristic error correction:  $\Delta w = \eta(y - \hat{y})x$ .
- **Mathematical Limitation:** The derivative is zero almost everywhere, making it non-differentiable.

---

## The Modern Neuron

- **Continuous Activation:** Functions like Sigmoid or Tanh provide a smooth, differentiable mapping.
- **Error Surface:** Creates a continuous "loss landscape" that can be navigated.
- **Gradient Descent:** Enables optimization by iterative methods.

# Modern Artificial Neuron

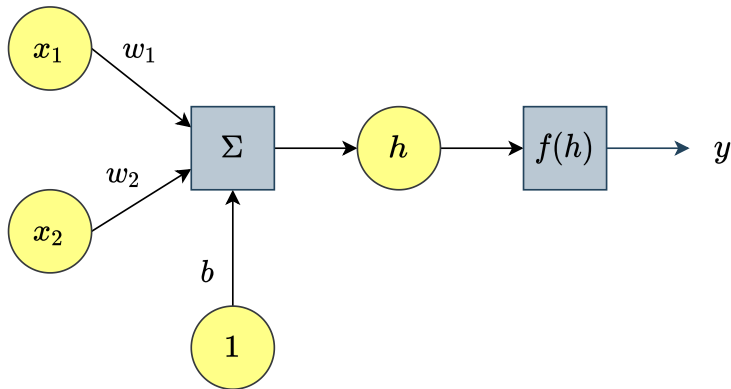
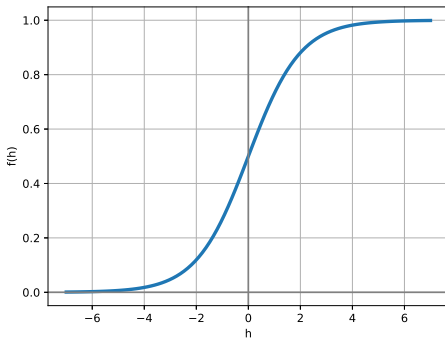


Figure: Diagram of a simple neural network (one perceptrón)

# Activation functions

- The activation function can be any function.
- A nice function is the sigmoid (logistic)

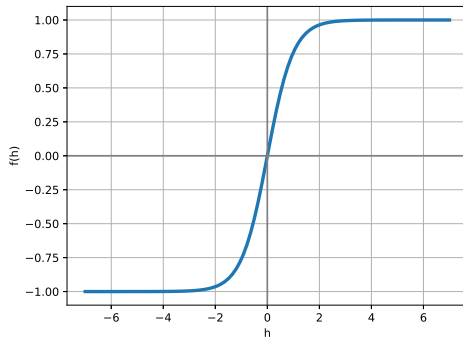
$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (1)$$



# Activation functions

- hyperbolic tangent.

$$f_{\tanh}(h) = \frac{e^h - e^{-h}}{e^h + e^{-h}} \quad (2)$$



# The simplest NN

$$\hat{y} = f(h) = \textit{sigmoid}\left(\sum_{i=1}^n w_i x_i + b\right) \quad (3)$$

- A simple NN does not offer advantage over linear regression models.
- Stacking nodes provides a powerful tool versus regression models.

# Evolution: Rosenblatt Perceptron vs. Modern Neuron

## Key Takeaway

The shift to continuous activations moved Neural Networks from simple logic gates to powerful function approximators trainable via calculus.

# Roadmap

- 1 Modern Artificial Neuron
- 2 Weight learning
- 3 Simple Network with multiple outputs
- 4 Practical aspects

## ¿Cómo aprendemos?



Figure: Ilustración de aprendizaje. <sup>2</sup>

- How wrong the predictions are?
- Sum of squared errors (SSE)<sup>3</sup>

$$E = \frac{1}{2} \sum_{\mu=1}^m [y^{\mu} - \hat{y}^{\mu}]^2 \quad (4)$$

where  $\hat{y}$  is the prediction,  $y$  is the true value and  $\mu$  the data row.

---

<sup>3</sup>\*divided by 2 for further derivation

- Prediction

$$\hat{y}^{\mu} = f \left( \sum_{i=1}^n w_i x_i^{\mu} + b \right) \quad (5)$$

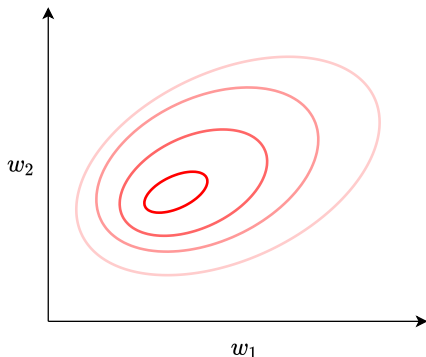
- The error depends on the weights

$$E(w_i) = \frac{1}{2} \sum_{\mu=1}^m \left[ y^{\mu} - f \left( \sum_{i=1}^n w_i x_i^{\mu} + b \right) \right]^2 \quad (6)$$

- Our goal is to find the weights that minimize the error.

# Gradient Descent

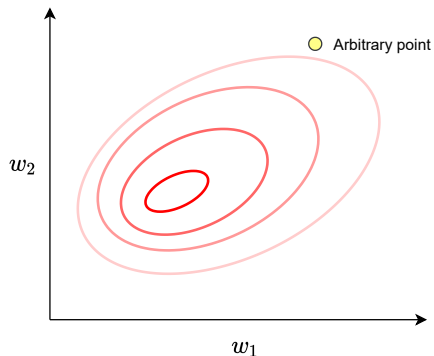
- At each step the error and the gradient are calculated, then the weights are calculated <sup>4</sup>.



<sup>4</sup>Rosenbloom, P. "The method of steepest descent." Proc Symp Appl Math. Vol. 6. 1956.

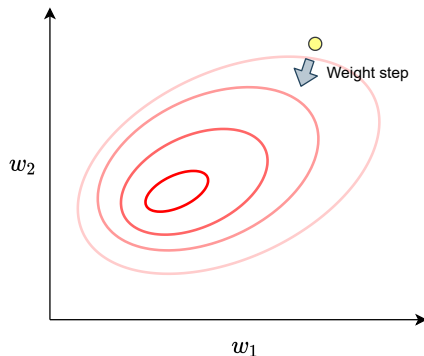
# Gradient Descent

- At each step the error and the gradient are calculated, then the weights are calculated.



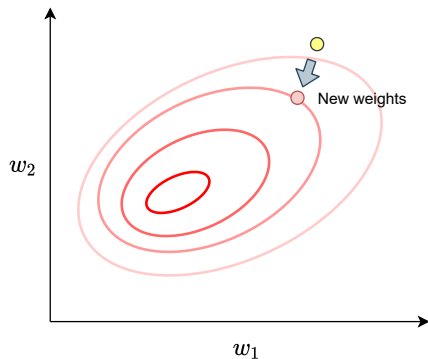
# Gradient Descent

- At each step the error and the gradient are calculated, then the weights are calculated.



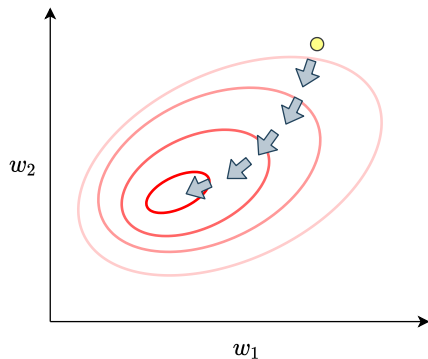
# Gradient Descent

- At each step the error and the gradient are calculated, then the weights are calculated.



# Gradient Descent

- At each step the error and the gradient are calculated, then the weights are calculated.



# Gradient descent

## Simplification Rule

- The gradient of the error function

$$\frac{\partial E}{\partial w_i} = -(y - \hat{y})f'(h)x_i \quad (7)$$

- Error term

$$\delta = (y - \hat{y})f'(h) = (y - \hat{y})f'(\sum_i w_i x_i) \quad (8)$$

- Weight step

$$\Delta w_i = \eta \delta x_i \quad (9)$$

- Weight update

$$w_i = w_i + \Delta w_i \quad (10)$$

# Activation functions

- Derivative of the Sigmoid

$$f(x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (11)$$

$$f' = f(1 - f) \quad (12)$$

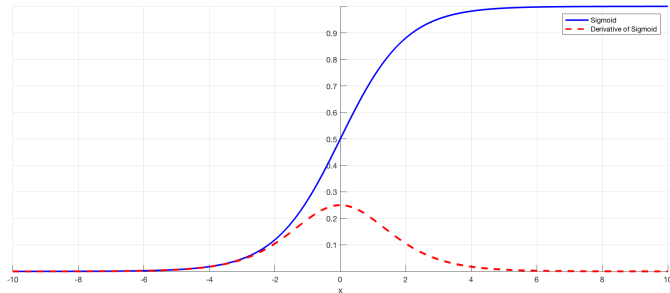


Figure: Sigmoid and its derivative

# Activation functions

- Derivative of the tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (13)$$

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x) \quad (14)$$

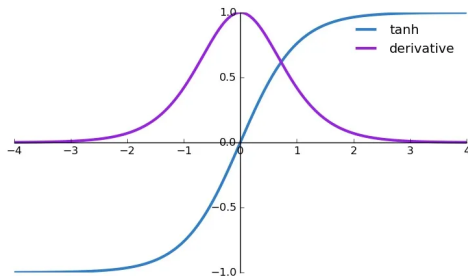


Figure: Hiperbolic tangent and its derivative

# Gradient descent algorithm

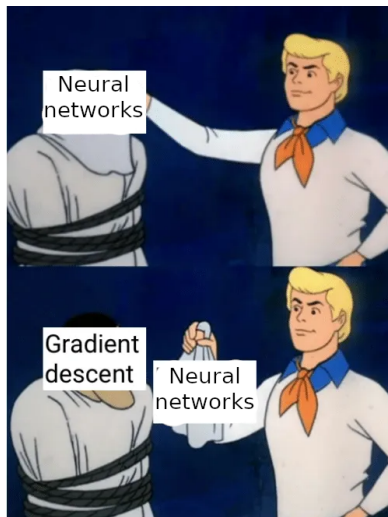
**Data:** Datos ( $Data$ ), número de registros ( $m$ ), tasa de aprendizaje ( $\eta$ ), épocas ( $epochs$ )

**Result:** Pesos entrenados ( $W$ )

```
Inicializar( $W$ );                                     /* Inicializar pesos */
for  $e = 1; epochs$  do
    Ceros( $\Delta W_i$ ) ;
    foreach  $(x, y) \in Data$  do
         $\hat{y} = f(h(x, W))$  ;                               /* Pase frontal */
         $\delta = (y - \hat{y})f'(h)$  ;                       /* Término de error */
        foreach  $w_i \in W$  do
             $\Delta w_i = \Delta w_i + \delta x_i$  ;           /* Incremento */
        end
    end
    foreach  $w_i \in W$  do
         $w_i = w_i + \frac{\eta}{m} \Delta w_i$  ;                 /* Actualización normalizada */
    end
end
return  $W$ 
```

## Algorithm 1: Descenso por gradiente

# Neural networks



# Roadmap

- 1 Modern Artificial Neuron
- 2 Weight learning
- 3 Simple Network with multiple outputs**
- 4 Practical aspects

# Multiple outputs

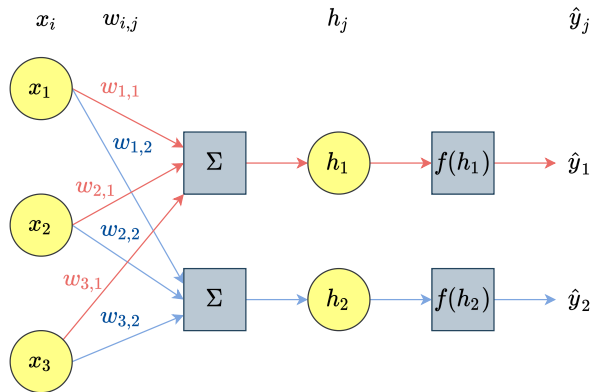


Figure: Neural network with multiple outputs

# Learning the weights for multiple outputs

- Prediction

$$\hat{y}_j^\mu = f \left( \sum_i w_{i,j} x_i + b \right) \quad (15)$$

- Sum of squared errors (SSE)

$$E = \frac{1}{2} \sum_{\mu} \sum_j \left[ y_j^\mu - \hat{y}_j^\mu \right]^2 \quad (16)$$

where  $\hat{y}$  is the prediction,  $y$  is the true value,  $j$  the output and  $\mu$  the data row.  
Substituting:

$$E = \frac{1}{2} \sum_{\mu} \sum_j \left[ y_j^\mu - f \left( \sum_i w_{i,j} x_i + b \right) \right]^2 \quad (17)$$

# Learning the weights for multiple outputs

- Error term

$$\delta_j = (y_j - \hat{y}_j)f'(h_j) \quad (18)$$

- Weight step

$$\Delta w_{i,j} = \eta \delta_j x_i \quad (19)$$

- Weight update

$$w_{i,j} = w_{i,j} + \Delta w_{i,j} \quad (20)$$

# Roadmap

- 1 Modern Artificial Neuron
- 2 Weight learning
- 3 Simple Network with multiple outputs
- 4 Practical aspects**

# Mean squared error

## Loss functions

- Avoids the divergence from the gradient descent provoked by a large data set.

$$E = \frac{1}{m} \sum_{\mu} (y^{\mu} - \hat{y}^{\mu})^2 \quad (21)$$

- $m$  is the number of examples

# Mean absolute error

## Loss functions

- Smaller penalization to large errors

$$E = \frac{1}{m} \sum_{\mu} |y^{\mu} - \hat{y}^{\mu}| \quad (22)$$

- $m$  is the number of examples

# Weights initialization

- Set bias ( $b$ ) to zero
- Zero initialization for weights is not successful
- Random numbers is better but be careful of large numbers

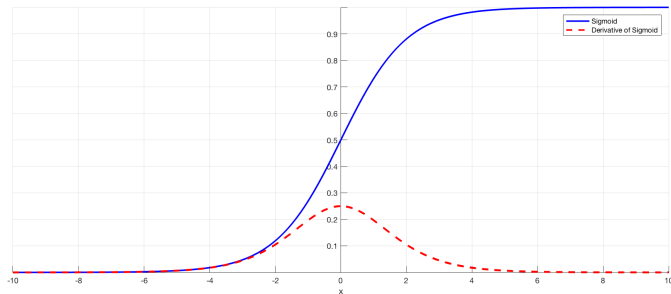


Figure: Sigmoid and its derivative

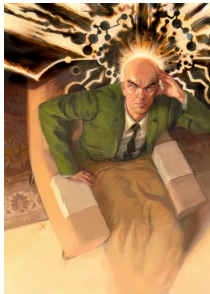
# Uniform Initialization

Assigns random values to the weights within a specific range.

$$W \sim \mathcal{U}(-a, a) \quad (23)$$

where  $a$  is a value that defines the boundaries of the range.

## Weights initialization (2)



- Xavier initialization <sup>5</sup>

$$W_{i,j} \sim U\left(-\frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}}\right) \quad (24)$$

where  $U(-a, a)$  is the uniform distribution in the interval  $(-a, a)$  and  $n$  is the number of input units.

---

<sup>5</sup>Glorot, Xavier, and Yoshua Bengio. "Understanding the difficulty of training deep feedforward neural networks." Proceedings of the thirteenth international conference on artificial intelligence and statistics. JMLR Workshop and Conference Proceedings, 2010.

## He (Kaiming) Initialization <sup>6</sup>

Specially designed for layers using the **ReLU** (Rectified Linear Unit) activation function or its variants.

- Addresses the fact that ReLU is zero for half of its domain.
- For a normal distribution, the weights are sampled from:

$$W \sim \mathcal{N}(0, \sigma^2) \quad \text{where} \quad \sigma = \sqrt{\frac{2}{n_{in}}} \quad (25)$$

- For a uniform distribution, the range is:

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}}\right) \quad (26)$$

---

<sup>6</sup>He, Kaiming, et al. "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification." Proceedings of the IEEE international conference on computer vision. 2015.

## Further readings

- WIDROW, B. (1995). Perceptrons, Adalines, and Backpropagation. The Handbook of Brain Theory and Neural Networks.

- Vasquez Gomez, Introducción a las redes neuronales, CIDETEC-IPN
- Goodfellow, Ian, et al. Deep learning. Vol. 1. No. 2. Cambridge: MIT press, 2016.
- Zhang, Aston, et al. Dive into deep learning. Cambridge University Press, 2023.
- Stewart, James, Daniel K. Clegg, and Saleem Watson. Calculus: early transcendentals. Cengage Learning, 2020.
- Buduma, Nithin, Nikhil Buduma, and Joe Papa. Fundamentals of deep learning: Designing next-generation machine intelligence algorithms. " O'Reilly Media, Inc.", 2022.